Lecture 5.2: Data Collection via APIs

AREC 570 / ECON 530 / Unit 5

Lauren Chenarides

Reading and sequence

Dominitz and Manski, "More Data or Better Data? A Statistical Decision Problem" (Dominitz and Manski 2017)

1. Survey the APIs available to economists
2. Retrieve four types of data in R
3. Check what each downloaded observation represents
4. Document a reproducible retrieval
5. Share the current data plan for each research project

APIs deliver secondary data

The organization responsible for the source determines

- What is measured
- Which people, places, or activities are covered
- When observations are collected and released
- How uncertainty, missing values, and revisions are recorded

An **application programming interface** provides a documented way for R (or other programming languages) to request selected records from that source

Working in R and RStudio

R is the programming language that sends each request and holds the returned data. **RStudio** is the application where you write R code in a script, run it, and see the results

Collecting data with code has three advantages over clicking through a website

- The script records exactly what was requested: place, period, and variables
- Rerunning the script repeats the request, for you or a coauthor
- Changing one value extends the request to more counties or years

Today we will run API requests in RStudio and try short exercises inside these slides

What happens when R calls an API

Step	What happens	Streamflow example
1. R function	You call a package function and list what you want	`read_waterdata_daily()` with a station, a parameter, and dates
2. Request	The package adds those **parameters** to the service's web address, called the **endpoint**	`api.waterdata.usgs.gov/...?monitoring_location_id=USGS-06752260&parameter_code=00060`
3. Data service	The agency's server finds the matching records	USGS Water Data
4. Response	The server returns the records in a **format** the service chooses: JSON or CSV for tables, GeoTIFF or NetCDF for grids	JSON text, one record per day
5. R object	The package reads that format and returns a data frame, raster, or map layer	183 rows, one per day

Response formats

Each service chooses the formats it offers. R packages read all of them

Format	What it holds	Today's examples	R reader
JSON	Records as labeled text	USGS Water Data; Census API; error messages	`jsonlite`, used inside `dataRetrieval` and `tidycensus`
CSV	Rows and columns as plain text	CDL acreage; NASA POWER through `nasapower`	`readr`
GeoTIFF (`.tif`)	One grid of values with map coordinates	Cropland Data Layer	`terra`
NetCDF (`.nc`)	Grids over space and time, with units and metadata	NASA POWER regional request; gridMET	`terra`, `ncdf4`
Shapefile, GeoPackage	Points, lines, and polygons	Census tract boundaries	`sf`

Tables often travel as JSON or CSV. Grids travel as GeoTIFF or NetCDF, because the same grid written as text would be far larger

Example: a JSON response becomes a data frame

The USGS service returns text like this for each day (simplified)

```
{"time": "2024-07-01", "value": 86.4, "unit_of_measure": "ft^3/s", "approval_status": "Approved"}
```

`dataRetrieval` reads the JSON and turns the records into rows of a data frame

```
  time       value unit_of_measure approval_status
  2024-07-01  86.4 ft^3/s          Approved
  2024-07-02 162   ft^3/s          Approved
```

You then work with the data frame; the JSON is never saved unless you save it

Which format did the server send?

- **Documentation** lists the formats and the parameter that picks one. NASA POWER takes `format=JSON`, `CSV`, or `NETCDF`
- **The response header** names the format in `Content-Type`: `application/json`, `text/csv`, `image/tiff`, `application/x-netcdf`
- **Saving the raw response** keeps the file exactly as sent, before any package converts it

The packages in today's examples choose a format for you. `nasapower`, for example, requests CSV

Getting access

Services use one of three arrangements

- **Open:** no registration, as with USGS Water Data, NASA POWER, and the World Bank
- **Agency key:** a free key from one agency, as with Census, BLS, BEA, FRED, and NASS
- **Shared key:** one api.data.gov key works across many federal services, including ERS and FoodData Central

A key is a personal credential. Store it in your user environment file and keep it out of shared code and repositories

The API landscape

Provider	Typical research uses
Census Bureau	Population, households, businesses, labor markets, trade
Other federal statistical agencies	Prices, income, employment, energy, finance
USDA	Production, prices, farm finances, food, soils, forests
Environment and natural resources	Water, weather, climate, land cover, pollution, hazards
International organizations	Cross-country development, agriculture, trade

Census Bureau: people and places

Source	Economists use it for	R	Key Required
American Community Survey	Income, poverty, housing, and commuting by tract or county	`tidycensus`	Yes
ACS Public Use Microdata	Person and household records	`tidycensus`	Yes
Decennial Census	Population counts and demographics	`tidycensus`	Yes
Population Estimates	Annual population, births, deaths, and migration	`tidycensus`	Yes
SAIPE and SAHIE	County poverty, median income, and uninsured rates	`censusapi`	Yes

Census API dataset list | Census developer guidance

Census Bureau: businesses and labor markets

Source	Economists use it for	R	Key Required
County and ZIP Business Patterns	Establishments, employment, and payroll by industry	`censusapi`	Yes
Nonemployer Statistics	Self-employment and small businesses by industry	`censusapi`	Yes
Economic Census and Annual Business Survey	Sales, ownership, and firm characteristics	`censusapi`	Yes
Quarterly Workforce Indicators	Hires, separations, and earnings by worker group	`censusapi`	Yes
LEHD Origin-Destination (LODES)	Where people live and work, by census block	`lehdr`	No

Census Bureau: trade, indicators, and geography

Source	Economists use it for	R	Key Required
International Trade	Monthly imports and exports by product and partner	`censusapi`	Yes
Economic Indicators	Retail sales, housing starts, and manufacturing	`censusapi`	Yes
TIGER/Line boundaries	Tract, county, and ZIP code tabulation area maps	`tigris`	No
Census Geocoder	Assigning addresses to tracts and blocks	`tidygeocoder`	No

Geographic identifiers and boundaries link nearly every source in this catalog

Federal statistics: prices, income, and labor

Source	Economists use it for	R	Key Required
Bureau of Labor Statistics	CPI, PPI, employment, and wages (QCEW, LAUS, OEWS)	`blsR`	Yes
Bureau of Economic Analysis	GDP and personal income by state and county	`bea.R`	Yes
FRED and ALFRED	Macro and regional series, including past vintages	`fredr`	Yes
IPUMS	Harmonized Census, ACS, and CPS microdata; NHGIS tables	`ipumsr`	Yes
HUD User	USPS ZIP code crosswalks and Fair Market Rents	`httr2`	Yes

BLS | BEA | FRED | IPUMS

Federal statistics: energy, finance, and spending

Source	Economists use it for	R	Key Required
Energy Information Administration	Energy prices, production, and consumption by state	`eia`	Yes
CFPB HMDA	Mortgage applications and lending decisions	`httr2`	No
USAspending	Federal grants, contracts, and payments by place	`httr2`	No
Treasury Fiscal Data	Federal debt, receipts, and interest rates	`httr2`	No
FDIC BankFind	Bank branches and financial reports	`httr2`	No

EIA Open Data

USDA: production and markets

Source	Economists use it for	R	Key Required
NASS Quick Stats	Acreage, yields, prices, inventories, and the Census of Agriculture	`rnassqs`	Yes
NASS Cropland Data Layer	Annual 30-meter crop and land-cover map	`CropScapeR`	No
AMS Market News	Cash grain, livestock, dairy, and specialty crop prices	`httr2`	Yes
FAS Open Data	Global supply and use (PSD), trade (GATS), export sales	`httr2`	Varies
ERS ARMS	Farm finances, costs of production, and practices	`httr2`	api.data.gov

Quick Stats API | MyMarketNews | ERS data APIs

USDA: food, soils, snow, and forests

Source	Economists use it for	R	Key Required
FoodData Central	Nutrient composition of foods	`httr2`	api.data.gov
NRCS Soil Data Access	Soil properties and productivity (SSURGO)	`soilDB`	No
NRCS Air and Water Database	Snowpack and soil moisture (SNOTEL, SCAN)	`soilDB`	No
Forest Inventory and Analysis	Forest area, volume, growth, and removals	`rFIA`	No

Soil Data Access

Environment: water, drought, and hazards

Source	Economists use it for	R	Key Required
USGS Water Data	Streamflow and groundwater levels	`dataRetrieval`	No
Water Quality Portal	Samples from federal, state, and tribal monitoring	`dataRetrieval`	No
U.S. Drought Monitor	Weekly drought categories by county	`httr2`	No
OpenFEMA	Disaster declarations, flood insurance claims, and assistance	`httr2`	No

USGS Water Data APIs | Drought Monitor services | OpenFEMA

Environment: weather and climate

Source	Economists use it for	R	Key Required
NOAA NCEI	Daily station weather records (GHCN-Daily)	`httr2`	No
PRISM	Gridded U.S. temperature and precipitation	`prism`	No
gridMET and Daymet	Daily gridded weather for exposure measures	`climateR`, `daymetr`	No
NASA POWER	Global daily weather and solar radiation	`nasapower`	No

NCEI data service | nasapower

Environment: land, pollution, and energy

Source	Economists use it for	R	Key Required
NASA Earthdata	MODIS and Landsat imagery: vegetation, land cover, fire	`earthdatalogin`, `rstac`	Account
Google Earth Engine	Processing large satellite archives	`rgee`	Account
EPA Air Quality System	Monitor-level air pollution	`RAQSAPI`	Yes
EPA ECHO	Facility permits, inspections, and enforcement	`echor`	No
EPA Envirofacts	Toxic releases (TRI) and greenhouse gas reporting	`httr2`	No
NREL developer APIs	Solar resource, utility rates, fueling stations	`httr2`	api.data.gov

International

Source	Economists use it for	R	Key Required
World Bank Indicators	Development indicators by country and year	`WDI`	No
FAOSTAT	Agricultural production, trade, prices, and land use	`FAOSTAT`	Varies
UN Comtrade	Bilateral trade by product	`comtradr`	Yes
OECD Data Explorer	Country statistics, including agricultural support	`rsdmx`	No
IMF	Financial statistics and commodity prices	`imf.data`	No
Eurostat	EU regional and sectoral statistics	`eurostat`	No

Sources without an API

Some providers distribute data only as files to download, or only to users who sign a license or data-use agreement

Access	Examples
File download	RMA Summary of Business (crop insurance by county and crop); IRS Statistics of Income (income by ZIP code, county migration); ERS data products such as the Food Environment Atlas; state water-rights and land records
License or agreement	Scanner and point-of-sale data; restricted federal microdata

If this is the case:

Download files with a short R script so the URL, file, and date are recorded

```r
url <- "https://www.irs.gov/pub/irs-soi/22zpallagi.csv"   # IRS SOI, ZIP code data, tax year 2022
download.file(url, "data/raw/irs_soi_zip_2022.csv", mode = "wb")
```

For licensed data, record the provider, version, delivery date, and license terms

APIs change and disappear

- NOAA moved station data from the Climate Data Online API to a new data service, and the `rnoaa` package left CRAN in 2024
- EPA removed EJScreen and its map services in February 2025; reconstructions are now hosted outside EPA
- USGS plans to retire the services behind the `readNWIS` functions in 2027

Save the raw response and the retrieval date so the data can be reproduced after a service changes

Examples

Today’s four examples

Catalog group	Source	Returned to R
USDA	Cropland Data Layer	Raster map or summary table
Environment: water	USGS Water Data	Daily time series
Environment: weather	NASA POWER	Daily weather table
Census Bureau	American Community Survey	Estimates, margins of error, and map geometry

The four sources differ in access, unit of observation, and geography.

Each example begins with a small request that can be inspected before the request is expanded

Two ways to run today's code

In RStudio. The *Request* slides show code that contacts each agency's server and downloads the 2024 data. You will run it from `api_examples.R` in the follow-along repository

In these slides. The *Try it* slides run R inside your web browser, with no installation

1. Open the HTML version of this lecture and wait for the *R ready* message
2. Select **Run Code**, change the value named in the instructions, and run again
3. Select **Start Over** to restore the original code

The browser version of R cannot install the API packages or contact most agency servers, so each *Try it* cell starts from rows of the real 2024 downloads typed into the cell. The PDF shows the code without running it

Follow-along repository

github.com/laurenchenarides/arec_570_demo

For today, download the Lecture 5.2 version as a ZIP file; Git is not needed yet

1. Download arec_570_demo-5.2-retrieve.zip and unzip it
2. Double-click `arec_570_demo.Rproj` to open the project in RStudio
3. Open `api_examples.R` and run one line at a time with **Ctrl+Enter** (**Cmd+Return** on a Mac)

The repository also holds the full API catalog (`docs/api-catalog.md`). Lectures 5.3 to 5.5 build on the same project

Install the R packages

Run once in the RStudio Console. The list matches the top of `api_examples.R`

```
install.packages(c(
  "CropScapeR", "dataRetrieval", "nasapower", "tidycensus",
  "ggplot2", "dplyr", "readr", "sf"
))
```

`api_examples.R` creates a `data/raw` folder inside the project and saves each download there

A retrieval workflow

1. Read the source documentation
2. Define the place, time period, variables, and unit of observation
3. Run a small request
4. Inspect column names, values, units, and missingness
5. Save the returned file or object
6. Record the request and retrieval date
7. Expand the request after the checks are complete

Example 1: land use

Question

Which crop and land-cover classes does the Cropland Data Layer assign in Larimer County in 2024?

Source

USDA NASS Cropland Data Layer through the CropScape service

Observation

Each raster cell has a modeled crop or land-cover class for a particular year

CropScape documentation | CropScapeR documentation

Request a Cropland Data Layer map

```
library(CropScapeR)

larimer_cdl <- GetCDLData(
  aoi = "08069",
  year = 2024,
  type = "f",
  save_path = "data/raw/cdl_larimer_2024.tif"
)

raster::plot(larimer_cdl)
```

`08069` is the five-digit Census code for Larimer County

The saved GeoTIFF contains a grid of land-cover codes and the information needed to locate those cells on a map

Use the land-use data

- Map crop locations
- Calculate acreage by crop class
- Compare annual crop patterns
- Define land-use exposure near water infrastructure or development

Try it: summarize land-cover records

Change `selected`, then run the cell. The acreage comes from the 2024 Larimer County download; “All other classes” combines the remaining 29 classes

R Code | Start Over | Run Code

```r
cdl <- data.frame(
  class = c("Evergreen Forest", "Grass/Pasture", "Shrubland",
            "Corn", "Alfalfa", "Winter Wheat", "All other classes"),
  acres = c(584635.7, 485338.1, 410812.1, 13266.7, 9022.6, 2386.3, 180286.1)
)
selected <- c("Corn", "Alfalfa", "Winter Wheat")
chosen <- subset(cdl, class %in% selected)
chosen$county_share <- round(chosen$acres / sum(cdl$acres), 4)
chosen
```

Before using the land-use data

Questions to answer from the CDL documentation

- How does the CDL define each crop class, and did the definitions change across years?
- How accurate is the classification for this crop and state? (NASS publishes accuracy tables by state)
- What do the 30-meter pixels and the map projection imply for small fields?
- How are double-cropped fields classified?
- Does the study boundary stay the same across years?

The CDL records modeled land-cover classes. Parcel ownership, zoning, and management practices require additional sources

Example 2: water

Question

What was the daily flow of the Cache la Poudre River during the 2024 growing season?

Source

USGS Water Data API

Observation

Each row reports a daily statistic for one parameter at one monitoring location

USGS monitoring location 06752260 | dataRetrieval tutorial

Request daily streamflow

```
1  library(dataRetrieval)
2
3  poudre_flow <- read_waterdata_daily(
4    monitoring_location_id = "USGS-06752260",
5    parameter_code = "00060",
6    time = c("2024-04-01", "2024-09-30")
7  )
8
9  names(poudre_flow)
10 head(poudre_flow)
```

`00060` is the USGS parameter code for discharge

The returned columns include date, value, unit, approval status, qualifiers, station geometry, and a time-series identifier

Try it: define a low-flow threshold

Change `threshold`, then run the cell. The values are daily mean discharge for July 1 to 14, 2024, from the streamflow download

R Code | Start Over | Run Code

```r
poudre_flow <- data.frame(
  date = as.Date("2024-07-01") + 0:13,
  flow = c(86.4, 162, 117, 68.2, 101, 92, 30.3, 7.66, 22.4, 35.7, 38.6, 51, 25.9, 27.1)
)
threshold <- 20
cat("Days below threshold:", sum(poudre_flow$flow < threshold))
plot(flow ~ date, poudre_flow, type = "l", xlab = "", ylab = "Daily discharge (cubic feet per second)")
abline(h = threshold, col = "#C4513A", lty = 2)
```

Before using the streamflow series

Questions to answer from the USGS record

- Is this the right monitoring location and parameter (00060, discharge)?
- What unit does the series use (cubic feet per second)?
- Which values are provisional or revised? In the 2024 download, 72 of 183 days carry the qualifier REVISED
- Are any dates missing?
- Do reservoirs, diversions, or return flows upstream affect the gauge?
- How does one gauge relate to the study area?

The gauge measures conditions at one point on a river network. A study of irrigation access may also require diversion service areas, water rights, or delivery records

Example 3: weather

Question

What temperature and precipitation conditions occurred near Fort Collins during the 2024 growing season?

Source

NASA Prediction Of Worldwide Energy Resources or POWER

Observation

Each row reports modeled weather conditions for a grid cell and day

NASA POWER documentation | nasapower documentation

Request daily weather

```
library(nasapower)

fort_collins_weather <- get_power(
  community = "AG",
  lonlat = c(-105.08, 40.58),
  pars = c("T2M_MAX", "T2M_MIN", "PRECTOTCORR"),
  temporal_api = "daily",
  dates = c("2024-04-01", "2024-09-30")
)

head(fort_collins_weather)
```

`AG` selects parameters prepared for agricultural applications

The three requested measures are maximum temperature, minimum temperature, and precipitation

Use the weather data

- Count high-temperature days
- Calculate growing degree days
- Sum growing-season precipitation
- Construct weather exposures for farms, counties, or markets

Try it: define weather exposures

Change `heat_threshold`, then run the cell. The values are daily maximum temperature and precipitation for July 1 to 14, 2024, from the weather download

R Code | Start Over | Run Code

```r
tmax <- c(33.71, 30.82, 33.09, 30.03, 29.01, 32.76, 23.65, 28.66, 32.02, 34.21, 36.76, 38.52, 37.90, 38.55)
precip <- c(3.08, 0.95, 0.03, 0.01, 0.02, 0.01, 2.94, 0.03, 0.26, 0.16, 0.04, 0, 0.05, 0.26)
heat_threshold <- 35
c(heat_days = sum(tmax >= heat_threshold),
  total_precip_mm = sum(precip))
plot(tmax, type = "b", xlab = "Day of July", ylab = "Maximum temperature (C)")
abline(h = heat_threshold, col = "#C4513A", lty = 2)
```

Before using the weather data

Questions to answer from the POWER documentation

- How large is the grid cell? (About 0.5° by 0.625°, roughly 55 by 53 km near Fort Collins)
- Which coordinates represent each farm, county, or market in the study?
- How is each variable defined, and in what unit?
- Which time standard defines a day? (POWER daily values use local solar time)
- Which data version was retrieved, and can values be revised?

Every location in a grid cell receives the same values, so conditions at individual fields can differ from the cell estimate

Extension: the weather data as a NetCDF grid

Question

How did maximum temperature vary across northern Colorado during the July 2024 heat wave?

Why a different request

The point request returns one grid cell. A *regional* request returns every cell in a box, over every day requested, in one NetCDF file

Request

NASA POWER, daily maximum temperature, 39.5 to 41.5 N and 104.5 to 106.5 W, July 10 to 16, 2024

Without an API: downloading by hand

From the POWER Data Access Viewer

1. Choose the user community (Agroclimatology) and the temporal level (daily)
2. Select a region on the map, or type the corner coordinates
3. Enter the start and end dates
4. Check each parameter: maximum temperature
5. Choose the output format (NetCDF) and submit
6. Download the file, move it into `data/raw`, and rename it
7. Write down every choice, so the file can be explained later

Another year, region, or variable means repeating all seven steps

With an API: one request in a script

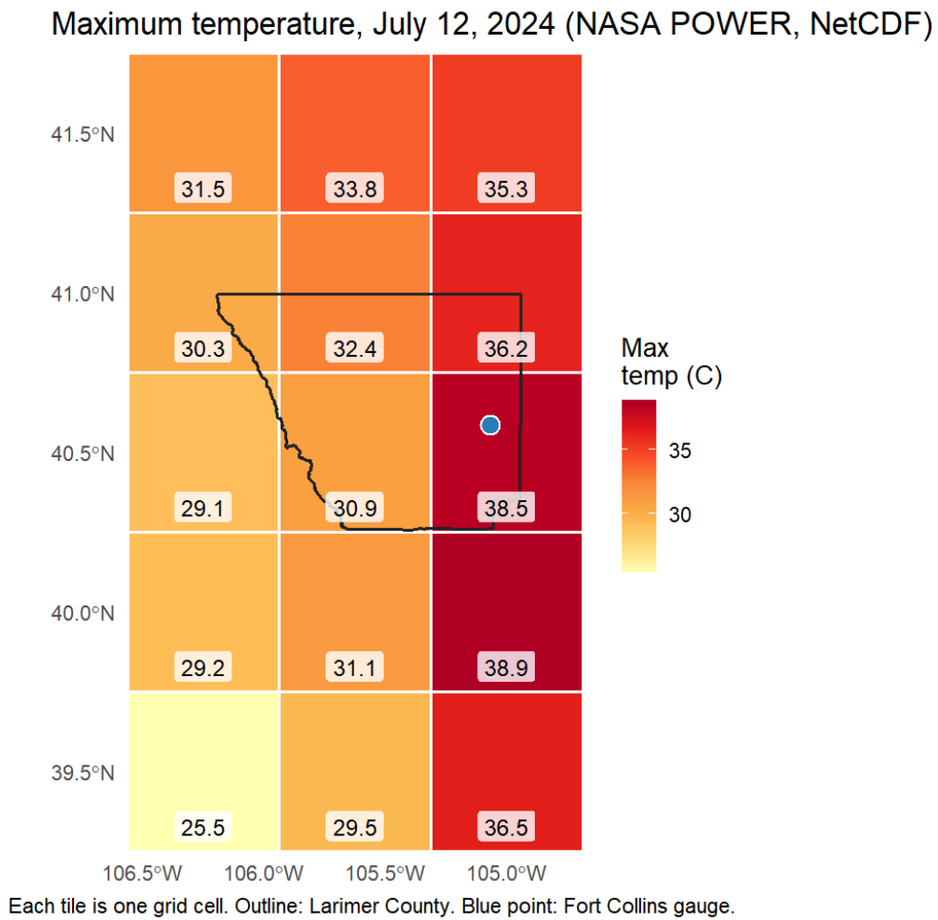
```
library(httr2)
library(terra)

request("https://power.larc.nasa.gov/api/temporal/daily/regional") |>
  req_url_query(parameters = "T2M_MAX", community = "AG",
                `latitude-min` = 39.5, `latitude-max` = 41.5,
                `longitude-min` = -106.5, `longitude-max` = -104.5,
                start = "20240710", end = "20240716", format = "NETCDF") |>
  req_perform(path = "data/raw/power_tmax_2024-07.nc")

tmax <- rast("data/raw/power_tmax_2024-07.nc")
tmax   # 15 grid cells (5 rows x 3 columns), 7 daily layers, degrees C
```

Each argument replaces a click. The file is saved exactly as the server sent it

What the NetCDF file holds



- Three dimensions: longitude, latitude, and day
- 15 cells x 7 days = 105 values in one 21 KB file
- Units (degrees C) and variable names travel inside the file
- `terra` reads each day as one layer of a raster

Fort Collins sits in one of the two hottest cells on July 12. The cells to its west, over the mountains, ran 8 to 9 degrees cooler

Download by hand or request through an API?

	Download by hand	API request in a script
Record of choices	Notes, if you write them	The code is the record
Another year, region, or variable	Repeat every step	Change one value, or loop
Consistency	A wrong box or date is easy to miss	The same request every time
A coauthor reproduces it	Follows your written steps	Runs the script
When a request fails	A website message	An error the script reports. A box smaller than 2 degrees returns a JSON message saying so
Best suited to	A first look at the data	Repeated or growing requests

Downloading by hand is reasonable for exploring a source. Once the data enter the analysis, write the request as code

Example 4: Census ACS

Question

How does median household income vary across census tracts in Larimer County?

Source

American Community Survey five-year estimates

Observation

Each row reports an estimate and margin of error for one variable and geographic area

Census API guidance | tidycensus documentation

Get a Census API key

The Census Bureau requires a key for every request

1. Complete the form at api.census.gov/data/key_signup.html
2. Open the email from the Census Bureau and select the activation link
3. Keep the 40-character key from that email for the next step

Until your key is active, skip section 4 of `api_examples.R`; sections 1 to 3 need no key

Store the Census API key

Open the user-level environment file from R

```
install.packages("usethis")
usethis::edit_r_environ()
```

Add one line, save the file, and restart R (**Session > Restart R**)

```
CENSUS_API_KEY=paste_your_key_here
```

Check that R can read it: `nzchar(Sys.getenv("CENSUS_API_KEY"))` returns `TRUE`

The file stays in your home directory, outside every project. Scripts read the key with `Sys.getenv("CENSUS_API_KEY")`

Request tract estimates and geometry

```r
library(tidycensus)

larimer_income <- get_acs(
  geography = "tract",
  variables = c(median_income = "B19013_001"),
  state = "CO",
  county = "Larimer",
  year = 2024,
  survey = "acs5",
  geometry = TRUE,
  key = Sys.getenv("CENSUS_API_KEY")
)

head(larimer_income)
```

The result contains a geographic identifier, name, estimate, margin of error, and tract geometry

Map the ACS estimates

```
library(ggplot2)

ggplot(larimer_income) +
  geom_sf(aes(fill = estimate), color = NA) +
  labs(
    fill = "Median income",
    title = "Median household income by census tract"
  )
```

The default margin of error describes a 90 percent confidence interval

$$\text{estimate} \pm \text{margin of error}$$

Try it: use the ACS margin of error

Change `relative_moe_limit`, then run the cell. The four tracts are illustrative values with the same columns as the ACS download

R Code | Start Over | Run Code

```r
acs <- data.frame(
  tract = c("001", "002", "003", "004"),
  estimate = c(68400, 52100, 79300, 44700),
  moe = c(4100, 8700, 5200, 9600)
)
relative_moe_limit <- 0.15
acs$relative_moe <- round(acs$moe / acs$estimate, 2)
acs$retain <- acs$relative_moe <= relative_moe_limit
acs
```

Before using the ACS estimates

Questions to answer from the ACS documentation

- The 2024 five-year estimate pools responses from 2020 to 2024. Does a pooled period suit the question?
- How large is each margin of error relative to its estimate?
- Which tracts have estimates too uncertain to use?
- Are geographic identifiers stored as text, so leading zeros survive?
- Which tract boundaries apply? (2020 boundaries for the 2024 release)
- Are dollar values adjusted to a common year when comparing periods?

ACS data describe the population and socioeconomic setting; exposure measures come from other sources

The unit changes across sources

Source	Unit returned by the request	Time	Geography
Cropland Data Layer	Raster cell	Annual	Grid
USGS Water Data	Gauge and day	Daily	Monitoring point
NASA POWER	Grid cell and day	Daily	Weather grid
Census ACS	Estimate and area	Five-year period	Census tract

A combined dataset requires a written rule for assigning pixels, gauges, weather cells, and census areas to the study units

Combining the downloads

For every join record

- Identifier used by each source
- Geographic crosswalk or spatial rule
- Time aggregation rule
- Number of observations before and after the join
- Unmatched locations and periods
- Treatment of revised, suppressed, and missing values

The combined analytic sample includes the observations that satisfy all source coverage and matching rules

Dominitz and Manski

Allocating the collection budget

Dominitz and Manski ask how to allocate a fixed budget across collection processes that differ in cost and information (Dominitz and Manski 2017)

API access reduces the time required to retrieve additional observations

Research time is also required to

- Verify definitions and units
- Examine coverage, missing values, flags, and uncertainty
- Select spatial and temporal aggregation rules
- Validate joins across sources
- Archive metadata and retrieval records

Question: How should a researcher allocate one additional week across these tasks?

Select the access method

Project need	Useful access method
A defined subset repeated across places or periods	API request
Most records in a fixed release	Bulk download
A few values needed once	Web table or manual download
Repeated data cleaning and analysis	Scripted retrieval with archived raw files

Choose the method after reviewing the source documentation update schedule and request limits

Document the retrieval

Keep these items with the project

- Source and dataset name
- Data documentation
- Retrieval script
- Parameters submitted through the script
- Retrieval date
- Raw files returned by the service
- Processing script and data dictionary
- Package and R versions
- Validation results

Store API keys in the user environment file and exclude that file from version control

Class share: your data plan

Write one line for each item, specific enough that a classmate could start the same search

1. **Outcome or exposure:** the behavior or condition you need to observe
2. **Approach:** primary, secondary, or combined collection
3. **Sources and access:** the two catalog sources you marked, with the access type (API, file download, or license) and whether a key is required
4. **Unit of observation:** what one row represents, such as county-year or gauge-day
5. **Open question:** the part of collection you are still working through
6. **Next step:** one concrete action and when you will take it

Share your plan in about one minute. Template: `docs/data-plan.md` in the follow-along repository

Example: the demo project’s data plan

Item	Demo project
Outcome or exposure	Low-flow days and heat days on the Cache la Poudre River, 2024 growing season
Approach	Secondary, combined from public APIs
Sources and access	USGS Water Data (API, no key); NASA POWER (API, no key)
Unit of observation	Day: gauge-day for streamflow, grid cell-day for weather
Open question	How to assign one gauge and one weather grid cell to farms across the county
Next step	This week, request 2015 to 2024 for the same gauge and count revised values by year

Before Lecture 5.3

On your laptop, complete Parts One and Two of docs/setup-rstudio.md

1. Install Git and restart RStudio
2. Create a GitHub account
3. Connect RStudio to GitHub with `usethis` and a personal access token
4. Request, activate, and store a Census API key

Bring the laptop that you used to complete this exercise with you to class.

Make sure `usethis::git_sitrep()` runs without errors and `nzchar(Sys.getenv("CENSUS_API_KEY"))` returns `TRUE`.

We will copy the demo repository into your GitHub account together in class.

Summary

APIs provide scripted access to secondary data

R packages translate requests and responses into familiar R objects

Source documentation defines the population, measures, timing, geography, and uncertainty

Small requests make the returned structure easier to inspect

Retrieval records support reproducibility

Data joins help define the final analytic sample

References

Dominitz, Jeff, and Charles F. Manski. 2017. “More Data or Better Data? A Statistical Decision Problem.” *The Review of Economic Studies* 84 (4): 1583–605. https://doi.org/10.1093/restud/rdx005.

Course navigation

- All lectures
- Schedule and preparation
- Course reading list
- Download this lecture as a PDF